

CheatCode DJ (CCDJ) — Developer Handoff

As of: v1.0.17004

Prepared for: Dr. Paul Wang / development team

Prepared by: Mike Stock (product owner) with Claude (AI development partner)

This document exists because a lot of the *why* behind this codebase currently lives only in a long conversation history between Mike and Claude, not in the code itself. Read this before diving into CheatCodeDJ.jsx — it'll save you time.

This also expands on one line from the SOW sent 7/30 — "protect proprietary AI logic by moving it server-side" — which deserved more than a bullet point (see Section 2), and covers what's changed since that document went out, including a full live gig test run the night before this handoff shipped.

One more thing, up front: this is written the way I'd actually talk to you about the app, not dressed up for a formal handoff. That means it says plainly where something's unresolved, where a call was made on gut feel instead of proof, and where there's a real bug we haven't cracked yet. I'd rather you walk in knowing the real state of things than find out the hard way three weeks in.

What I'm handing you

- CheatCodeDJ.jsx — current application source, v1.0.17004
- Library mapping CSV (Crates.csv) — real sample library data for development and testing (see Section 3 for the schema)
- This developer handoff
- Available on request: the 9/2/26 CCDJ session-history JSON and the corresponding Serato history export (see Section 9)

1. What CCDJ Is

CCDJ is an AI-assisted DJ tool with three core recommendation engines:

- **Track Cheat** — drop a track, get 10 AI-reasoned recommendations for what to play next. Supports "daisy chaining" (drag a result back in as the new seed to keep a streak going).
- **Playlist Cheat** — import a Spotify playlist or plain-text track list, get each song matched against your own local library.
- **Crate Cheat** — describe a vibe in plain language ("2k bombs," "wedding reception energy"), get a full curated crate built from your library.

Plus a **Library** tab that ingests a DJ's music library (tested at 75,000+ tracks) and runs a background **Artist Era Resolver** that identifies real artists, their active era, and known aliases — this powers a lot of the matching intelligence elsewhere in the app.

2. CRITICAL — Read This Before Planning Any Deployment

CCDJ currently only works because it runs inside Claude.ai's artifact environment. There is no backend server, and no Anthropic API key anywhere in the code. Every AI call (`fetch("https://api.anthropic.com/v1/messages", ...)`) works *only* because Anthropic's artifact-hosting infrastructure authenticates these requests automatically — but only when the code is actually running inside a Claude.ai artifact, under Mike's own Claude account.

What this means concretely:

- The moment this becomes a standalone Mac app, **every single AI call will fail outright** — there's no key, no auth, nothing to fall back on. This isn't a partial-functionality issue; the AI features are completely dead outside the artifact context.
- **A real backend server is not optional** — it's required. The server holds your own Anthropic API key; the app talks to your server; your server is the only thing that ever talks to Anthropic directly. (Do NOT embed an API key directly in a distributed Mac app — that's a well-known, serious security mistake; anyone could extract it and run up charges on your account.)
- Right now, **every AI call from every user draws against Mike's own personal Claude usage** — there's no isolation between users because there's only ever been one "user" (Mike, testing).
- **The server isn't just plumbing — it's what protects the actual product.** The real value here isn't "an app that calls Claude." It's the specific reasoning: how Track Cheat weighs BPM/key/vibe/energy, and features like Bridge Mode and Style Drift Detection (Section 5). All of that logic currently sits in plain client-side JavaScript — anyone with basic dev tools could open the app right now and read exactly how it thinks. Moving that server-side means the client only ever sends "give me recommendations" and gets results back. The actual reasoning never ships to the user's machine. That's the whole moat, and right now it isn't protected at all.
- **Practical upside of a real server, worth saying plainly:** once it's holding Mike's own API key and proxying every call, end users never need their own Claude account or key — they just use the app, and it all runs invisibly through Mike's backend.

Cost planning — needs a real number before anything gets built on top of it

Earlier internal estimates of per-user AI cost were based on assumptions about how much data gets sent per API call, not a real measurement. Checked directly against the actual code: every Track Cheat/Crate Cheat call sends the model a candidate pool of library tracks as part of the prompt (see `buildSmartCtx` in Architecture below). At the pool size in use through most of testing (800 tracks), that's roughly 12,000-14,000 input tokens per call — a meaningfully larger prompt than earlier estimates assumed.

The fix, live-tested the same night this was found: that 800-track pool was far more than the model needs to pick 20 good tracks — it's already pre-sorted by genre/era/BPM relevance before being cut off. Cut to 200 (v1.0.17002) and confirmed live at a real gig immediately after: Crate Cheat build time dropped from 13 seconds to 8 seconds, with no quality regression noticed across any tab that same night (see Section 9 for full validation data). A meaningfully smaller prompt should also reduce per-call API cost — by how much needs to be measured, not assumed.

What's actually needed next: run the real prompt templates through Anthropic's token counter, get actual input/output token counts per call type, and build a real cost-per-user model from that — not another estimate. Pricing and margin decisions should wait on that real number rather than any guess, including earlier ones.

Bottom line for planning: treat "build and host our own API proxy server" as a first-class, required piece of scope — not a nice-to-have, not a phase-2 item.

Do not begin by simply wrapping the existing JSX in Electron and treating that as the production architecture. The current artifact is the functional prototype and behavioral reference — it proves the app works, not how it should be built. Productionization requires separating the protected AI/recommendation logic from the client *first*. This is the single most consequential architectural instruction in this document.

3. Architecture Overview

- **Single React component**, currently ~5,500 lines in one file (`CheatCodeDJ.jsx`). Runs as a Claude artifact — no build step, no separate backend, no database. All state is in-memory + `window.storage` (a simple key-value persistence layer Claude artifacts provide).
- **No routing** — a single tab state integer (0-4) switches between Track Cheat / Playlist Cheat / Crate Cheat / Library / Account, all rendered conditionally in one big JSX tree.
- **Library data** lives in `libraryRef` (a React ref, not state — intentional, since the library can be tens of thousands of tracks and doesn't need to trigger re-renders on every access).
- **Session history** (used for daisy-chain analysis, discussed below) persists via `window.storage`, capped at the last 300 entries client-side.
- `buildSmartCtx`` is the shared candidate-pool builder behind Track Cheat's main fetch, both retry paths, and Crate Cheat's default build — one function, one place the pool-size cap lives (currently 200, split 150/50 across priority tiers — see the v1.0.17002 comment directly above it in the code for the full reasoning on why the internal split is scaled proportionally, not just the outer number).

Performance notes

Several matching functions (Playlist Cheat's library matcher, Crate Cheat's `matchCuratedTracksInLibrary`) were optimized by caching per-track normalized data directly on the track objects (e.g., `track.__crateMatch`), rather than recomputing string normalization from scratch for every comparison. This took Playlist Cheat from ~12 seconds to ~2 seconds on a 75k-track library. **Track Cheat has not received the same treatment yet** — lower priority now than it was, since the candidate-pool cut (below) already delivered a real speed win across both Track Cheat and Crate Cheat without it.

The candidate-pool cut from 800→200 tracks (v1.0.17002) is a separate, second speed/cost lever from the caching above — smaller prompt, less for the model to read, live-confirmed faster (13s→8s on Crate Cheat) the same night it shipped.

Library CSV format (the "mapping file")

`Crates.csv`, included with this handoff, is one real export from Mike's own library — the same format all his library files use. Parsing is generic, not hardcoded to a fixed schema: `parseCSV` reads whatever the header row contains, lowercases each header name, and builds one object per row keyed by those lowercased names — so a column add/remove/rename on Mike's end doesn't require a code change, it just changes which keys exist on each track object.

The columns actually present in this file, and how the rest of the app uses them: title, artist, bpm, key (Camelot notation, e.g. 8A), genre, playCount, duration, year, energy, danceability, location (full file path — this is what actually gets handed to Serato/the OS when a track is dragged out), dateAdded. A row is only kept at all if it has a title, location, or filename value — otherwise it's silently dropped as blank/junk.

One important gotcha, exactly why this matters: header casing isn't normalized beyond lowercasing — playCount (camelCase, no space) becomes the key `t.playcount`, not `t["play count"]`. That mismatch was a real bug found and fixed the same night this handoff shipped (see Known Issues #6) — worth being deliberate about exact key names anywhere new code reads a field off a track object, rather than assuming a "natural-looking" key.

4. Known Open Issues

#	Issue	Severity	Notes
1	Track Cheat has no per-track caching optimization	Low priority, lower than before	Crate Cheat and Playlist Cheat got this treatment; Track Cheat didn't. The candidate-pool cut already delivered most of the available speed win without it — still worth porting eventually, just less urgent.
2	Zero-result bug	Low priority, no repro since	Observed once in real gig data (8/26/26). Session history logs resultStatus specifically to help catch this if it recurs. A full live gig on 9/2/26 — 47 real fetches, every trigger type firing — produced zero errors and zero empty results. Not confirmed fixed, just hasn't recurred; keep the logging in place.
3	Playlist Cheat build button visual state	Cosmetic, deprioritized	The button doesn't reliably show a gray/loading state during a build. Functionally harmless.
4	Deck Watcher toggle is non-functional	By design, not a bug	Requires native OS-level Serato integration — see Section 8. Can't be built until CCDJ is a real desktop app.
5	Artist Era Resolver — unexplained slow/stalled batches	Open, unresolved	See Section 6.
6	Play count always reading as 0 in AI context	Fixed, v1.0.17004	Found during final review before this handoff: every AI-facing context string (buildSmartCtx and ~9 similar blocks) read play count as <code>t["play count"]</code> (with a space), but CSV headers like <code>playCount</code> parse down to <code>t.playcount</code> (no space) — these never matched, so the model has likely been seeing "0" for every track's play count on every single call. A few other spots (client-side sort/filter) already had the correct <code> t.playcount</code> fallback — this exact mismatch was caught once before, just never applied here. Fixed: same fallback added to all 10 affected spots.
7	Crate Cheat V2/"Curate & Match" Beta	Removed, v1.0.17003	Was a two-step "invent real songs from general knowledge, then match against library" approach, offered as an off-by-default toggle alongside the default library-first approach. Removed entirely after live gig testing showed the default path already producing exactly the desired quality, using real library-specific edits (custom CDJ remixes, mashups, DJ-made blends) that V2's own-knowledge-first approach structurally could never have named in the first place — it can only know the <i>original</i> song, not a specific DJ's custom edit of it. Not a bug fix, a product decision: two systems weren't worth maintaining once one was clearly, structurally better suited to this use case.

Dead code: `pickDiverseWithAIAsync` and `matchSpotifyStyleTracksAI` — the *original* Playlist Cheat picker, superseded by faster local matching. Confirmed unused (zero call sites), clearly marked in the code. Safe to delete — left in place intentionally ahead of handoff rather than risk breaking something with an unnecessary edit.

5. Design Philosophy — Read Before Touching Track Cheat's Reasoning

Track Cheat's daisy-chain reasoning has two non-obvious features layered on top of normal recommendation logic, both now validated against a full real gig (Section 9), not just design intent.

Bridge Mode

What triggers it: the DJ breaks their own daisy chain by dropping in a track that didn't come from Track Cheat's own suggestions (an external drop, or dragging a result from a different tab), **while a chain was already actively running**. A fresh, unchained seed drop never triggers this — see the exact condition (`!isDaisyChainResult && daisyChainDepth>0`) in `handleWindowDrop`.

What it does: the *next* batch of results is a deliberate hybrid — roughly half continuing the new direction normally, half genuine attempts to bridge back toward the vibe the DJ just left (using half-time/double-time BPM relationships, or genre-adjacent tracks). The normal BPM constraint is explicitly suspended for bridge candidates *only*, for that one batch. Results are tagged (camp: "bridge" | "new") so the UI can show a small, non-blocking indicator.

The key design principle: there's no popup, no question ever asked out loud. Whatever the DJ drags next from that hybrid batch *is* the answer — pick a bridge track, get one more hybrid round leaning further toward reconnection; pick a new-vibe track, the swerve is treated as confirmed and complete.

Live-validated 9/2/26: 5 real bridge swerves in one gig, all landing exactly where the timestamps said they should against the actual Serato history — clean, no bad triggers.

Open question, not yet resolved: should a swerve that happens right after the old vibe was already failing be treated the same as a swerve that happens mid-*success*? Right now both are handled identically. This is a live design question — don't "fix" this without discussing it first.

Style Drift Detection

What triggers it: *without* any external drop — the DJ's own picks *from Track Cheat's own results* start trending toward a genre crossover.

How it's detected: every Track Cheat result carries a passive lane tag ("pure", "crossover", or "native"), determined by the model reading remix/version-tag text as a genre-crossover signal. When nothing is drifting yet, this tagging **must not influence which tracks it picks** — pure classification only, a deliberate safeguard against recommendation quality quietly degrading.

What happens once drift is detected: the first non-"pure" pick triggers a true three-way hybrid batch (pure / crossover / native), same non-blocking-tag philosophy as Bridge Mode.

Live-validated 9/2/26: fired 15 times in one gig — the single most active trigger type of the night after plain daisy-chaining — and every one produced a genuinely diverse, well-tagged batch.

Genre-blend (positive Vibe slider) also uses this same lane-tag system, live-tested the same night across three different seed genres — tagging rendered correctly and consistently (native vs. crossover, correctly matched to actual remix-signal text) in every case. One soft observation, not a bug: pushed from a rock seed, the "different genre" picks it found were a noticeably milder flip (still broadly rock-adjacent) than the same test run from a hip-hop or EDM seed. Worth knowing, not something to chase without more data.

Both features share a philosophy, not a mechanism: detect a real signal → respond with genuine hybrid options → read the DJ's next pick as the answer, never ask directly. If you're adding a third trigger of this kind (see Section 7 — a real candidate for one already exists), follow the same shape.

6. Model Choice — Artist Era Resolver (Sonnet, Confirmed) — Plus an Open Performance Mystery

Early in development, the two **Artist Era Resolver batch functions** (`resolveArtistEraBatch` and `resolveTitleEraBatch`) were briefly switched from Claude Sonnet to Claude Haiku to reduce cost/speed on library import. **Tested against a real import and reverted back to Sonnet** — no clear speed improvement, and accuracy on this task (especially `resolveTitleEraBatch`'s harder inference work) was judged to matter more than the savings. A real side-by-side comparison (same library, both models, comparing resolved-artist counts and alias accuracy) would be the right way to actually settle this if it's revisited.

All AI calls in the app currently run on Sonnet.

Open, unresolved performance issue: the resolver has been observed crawling far slower than expected on a real 75k-track library (one measured stretch: 34 artists resolved in 69 minutes). Confirmed to persist after the Haiku revert (ruling out model choice) and with zero API errors logged (ruling out rate-limiting). Root cause still unknown — worth investigating whether it's genuinely a long tail of obscure/unrecognizable names getting rejected batch after batch (expected, not a bug) versus something else.

Also worth knowing: this resolver depends on staying mounted continuously in the browser tab. Navigating away unmounts the artifact and kills any in-flight batch — total resolved-count progress persists via `window.storage`, but whatever batch was mid-request is lost, and the run restarts from batch 0 on return. A production backend job shouldn't have this constraint.

7. Real Ideas Deferred to a Future Update — Not Built Yet, Specced Here on Purpose

These came directly out of live gig use on 9/2/26 and are genuinely good ideas, deliberately **not** built into tonight's ship — they need to interact correctly with Bridge Mode and Style Drift (Section 5), which just proved themselves clean in real conditions, and that's not something to risk on an untested same-night addition. Plan: build and test live at the next gig, ship to developers once proven, same process used for the pool-size cut.

Drag-as-signal: right now, dragging a Track Cheat result (`handleResultDragStart`) already logs the event (`logOutcome(..., "dragged")`), records play history, and excludes it from future suggestions — but it does **not** feed forward into what the *next* batch looks like. The idea: every single drag should nudge an internal directional signal — no minimum threshold before it counts, confidence should scale naturally with volume and consistency of drags in the same direction, not gate on an arbitrary count. A real reseed still confirms and locks in whatever direction the drags were already pointing toward.

Do not implement this by simply making plain drags increment `daisyChainDepth``. That variable is specifically what Bridge Mode's trigger condition depends on (`daisyChainDepth>0` meaning "a chain of actual reseeds is running") — repurposing it for raw drag-counting would make Bridge Mode misfire on a genuinely fresh seed drop that happens to follow an earlier unrelated drag. This needs its own separate signal, not reuse of an existing load-bearing one.

Serato drag integration, both directions: dragging a result from CCDJ into Serato should count as advancing the daisy chain the same way dragging it back into CCDJ's own seed area does — right now the app has zero visibility into Serato at all (same root limitation as the Deck Watcher toggle in Section 4), so this requires real native integration, not a browser-side fix. Confirmed explicit requirement: drag-and-drop needs to work **both directions** — CCDJ into Serato, and Serato into CCDJ, from anywhere Serato allows a file to be dragged from (deck/player, crates, browser panel). Not just one-way.

Planned next live tests (before this reaches developers as anything more than a spec): two rounds of testing at upcoming gigs — (1) building and testing the drag-as-signal + daisy-chain-depth idea above, and (2) dropping the candidate pool further, from 200 to 100, to see how far the cost/speed win from Section 2 can go before quality suffers. Expected results by the end of the Friday gig, possibly the Saturday gig if Friday doesn't give a clean enough read (lower app usage during Saturday's band-break sets).

How updates after this handoff will be delivered: the GitHub repository should be treated as the working source of truth going forward. Mike may continue developing and live-testing changes between developer updates; those changes will be committed/uploaded as complete, tested versions of `CheatCodeDJ.jsx`. Every meaningful change already carries a dated, reasoned comment at the point it was made (see the v1.0.17002/17003/17004 comments in the code as examples), so Git history/diffs should make it straightforward to see exactly what changed and why — better practice than emailing full files back and forth, and it avoids anyone wondering three weeks from now which copy is actually current.

8. Native App Requirements — Confirmed, Not Yet Built

Window sizing/placement: launches at a default size and position (developer-configured), user can freely resize/move after that, always remembers the last size/position used across launches, and "reset to default" returns to that exact spec. No multiple auto-placement options needed — one default is enough. Doesn't conflict with anything in the SOW (Phase 8 only mentions "minimize to compact overlay," deliberately without a fixed location).

Minimum window size: standard Electron `minWidth/minHeight` constraint. Likely the same number as the default size — Mike to confirm actual pixel dimensions for both when ready.

No scaling exists yet. Checked directly: every font size, button size, padding, and icon dimension in the current code is a hardcoded fixed pixel value, scattered across hundreds of individual style declarations — nothing ties any of it to window size. If the app needs to scale smoothly as the window resizes, the practical approach is a single global scale factor applied once at the window-chrome level (real in Electron via actual OS resize events), not manually converting hundreds of individual values by hand.

Themed logo assets: current code only has the logo embedded as static base64 images (one dark-mode version, one light-mode background variant) — no mechanism for the logo to change with the selected theme color. Mike will supply a full set of themed logo image variants (sunglasses color matching each theme color — 8 total including the default green) once developers are ready for the asset files; ask him directly when that point is reached, in whatever dimensions are needed.

9. Live Gig Validation — 9/2/26

Full night of real use, cross-referenced against the actual Serato play history and the exported CCDJ session log — not a synthetic test. Numbers:

- **47 real Track Cheat fetches** (3 additional test seeds — 50 Cent, Journey, Avicii — excluded from all of this, used only to sanity-check Style Drift/genre-blend behavior before the gig started)
- **Every trigger type in the design fired naturally, unprompted:** 19 daisyChain, 15 styleDrift, 5 bridgeSwerve, 3 manualRetry, 3 freshSeed, 2 likedAsSeed
- **Zero errors, zero empty results**, the entire night
- **106 tracks dragged out of 459 shown (~23% drag rate)**
- **~79% of dragged tracks (84 of 106) showed up somewhere in the actual Serato play history** by song title (sometimes the exact edit, sometimes a different edit of the same underlying song) — a genuinely strong real-world signal that drag events reflect real intent, not noise (see Section 7's drag-as-signal idea, directly motivated by this finding)
- **Candidate-pool cut (800→200, v1.0.17002) confirmed live:** Crate Cheat build time 13s→8s, no quality regression noticed across Track Cheat, Crate Cheat, or Playlist Cheat testing that same night

Raw data from this night (the session history export and the corresponding Serato history export) isn't included in this upload, but exists and is available on request if it'd be useful to look at directly — the cross-referencing above was done directly against those two files.

10. Session History Data Schema (for analytics/data work)

Every Track Cheat fetch is logged to sessionHistory (persisted via window.storage, exportable as JSON/CSV from the History panel):

```
{
  id, ts, sessionId, sessionTag, // sessionTag: optional free-text label, e.g. "wedding"
  chainId, chainDepth, // chainId is STABLE across a whole daisy-chain streak —
                        // filter to one chainId + sort by ts to reconstruct a
                        // full chain from an export
  seed: { name, bpm, key, genre, year },
  vibeOffset, energyOffset,
  trigger, // "freshSeed" | "daisyChain" | "bridgeSwerve" |
           // "styleDrift" | "manualRetry" | "search" | "likedAsSeed"
  bridgeInfo, // non-null only when Bridge Mode was active for this fetch
  driftInfo, // non-null only when Style Drift was active
  resultStatus, // "ok" | "empty" | "error" | "cachedFallback"
  errorReason, // populated when resultStatus is "error"
  results: [
    { name, bpm, key, genre, year, outcome, // outcome: "shown" | "dragged" | "up" | "down"
      camp, // "bridge" | "new" — only present during Bridge Mode
      lane, crossoverGenre // "pure" | "crossover" | "native" — present when tagged
    }
  ],
  note // free-text DJ annotation, added after the fact via the
        // History panel's per-entry note feature
}
```

11. Questions?

This document was built by an AI development partner (Claude) working directly with the product owner over an extended series of sessions, including one full live gig used specifically to validate everything in Section 9. If something in the code doesn't match what's described here, or a design decision seems unexplained, the conversation history contains the full reasoning — ask Mike, he has it.